

10-Day Prototype Execution Plan

SIH 2026 — ISRO: On-device Visual Perception for Lightweight Browser Agents

Goal: Internal College Selection Round

SECTION 1 — PROTOTYPE TARGET

What must be working by Day 10?

One sentence: An agent opens a browser page containing PII, receives a task via a simple UI, uses a local VLM to visually ground the target element, executes the browser action, and shows original vs. redacted screenshots side by side — with a note confirming PII never left the device.

MUST-HAVE DEMO (non-negotiable)

Step	What the judge sees
1	A test webpage opens in a real browser — it has a login form with username, password, email fields and a face photo
2	A task is typed into a simple control panel: e.g., "Click the Login button"
3	The agent captures a screenshot and sends it to the local VLM
4	The VLM returns: {element: "Login button", bbox: [x, y, w, h]}
5	The browser agent executes <code>click(x, y)</code> on the real browser
6	A new screenshot is taken — shows the page changed (login processed)
7	The PII scanner runs on the original screenshot → highlights password field, email, face
8	A side-by-side image appears: Original (PII visible) vs Sanitized (PII masked/blurred)
9	A simple text note shows: "Sanitized screenshot sent to remote model. Raw PII stayed on device."
10	A box shows: Local inference time

NICE TO HAVE (build only if core is solid by Day 7)

- Remote model actually called (not mocked) — receives sanitized screenshot, returns reasoning text shown on panel
- Face detection + blur working (not just rectangle mask)
- 2nd task example: "Find the Search button and click it" on a different test page
- VRAM meter updating live during inference

DO NOT BUILD IN THESE 10 DAYS

- Multi-step autonomous task planning
- Persistent agent memory / conversation history
- User authentication or login flows (for the demo app itself)
- Any database
- Production API security (rate limiting, JWT, etc.)
- Custom model fine-tuning
- Complex frontend with animations
- More than 2 test pages
- CI/CD pipeline
- Docker containerization

Final Demo Flow (reference for all 10 days)

```
[Control Panel]: Task = "Click the Login button"
↓
[Browser opens test_login.html - shows form with PII]
↓
[Screenshot captured locally]
↓
[Local VLM]: screenshot + "where is the Login button?"
→ returns: {bbox: [342, 280, 120, 40], confidence: 0.91}
↓
[Browser]: click(342 + 60, 280 + 20) ← center of bbox
↓
[Verification screenshot]: page shows "Logged in!" or similar
↓
[PII Scanner on original screenshot]:
- password field → black rectangle
- email field → black rectangle
- face photo → blurred
↓
[Side-by-side display]: Original | Sanitized
↓
[Text panel]: "Raw screenshot NOT transmitted.
              Sanitized version only → remote model (MOCKED)"
↓
[Metrics]: VLM inference: 420ms | Redaction: 45ms | VRAM: 3.8 GB
```

SECTION 2 — TEAM RESPONSIBILITIES

Member	Learn (specific, limited)	Build	Output by Day 10	Deadline
Ayush	Playwright: screenshot, click(x,y), goto, accessibility_snapshot. FastAPI: 2 endpoints + WebSocket. Python async basics.	Agent loop (observe→ground→act→verify). Control panel UI (simple HTML). Integration glue.	Running agent loop + demo UI	Day 5 (loop), Day 8 (UI)
Shaurya	llama.cpp: download, compile, llama-server flags (--model, --mmaproject, --n-gpu-layers, --port). OpenAI-compatible API format. nvidia-smi for VRAM.	Local VLM server running Qwen3.5-4B Q4_K_M. Python client wrapper. Verification (before/after screenshot compare).	VLM server responding to image+prompt in <600ms	Day 3 (server up), Day 6 (optimized)
Aditi	What a VLM does (image → tokens → text). What grounding means. Bounding box formats (x,y,w,h vs x1,y1,x2,y2, normalized vs pixel). How to write a grounding prompt.	Test 2–3 VLM prompt strategies. Find the best prompt for "locate element X" → returns valid bbox. Document best prompt + accuracy.	Grounding prompt that correctly locates elements in 8/10 test images	Day 4
Abhishek	Python re module basics. DOM attribute reading (Playwright). OpenCV rectangle + Gaussian blur. MediaPipe face detection API (3 lines of code).	PII detector: DOM scan (password/email fields) + regex (email, phone, Aadhaar pattern) + face detection. Redaction: mask fields, blur face.	redact(screenshot, dom) → sanitized screenshot	Day 6
Himanshu	HTTP request/response basics. What JSON looks like. How to explain data flow ("what moves where and why"). Mermaid diagram syntax.	Simple API spec (2–3 endpoints documented). Data flow diagram. Architecture slide for PPT. Help Ayush shape the FastAPI response format.	API spec doc + 2 Mermaid diagrams (flow + architecture)	Day 5 (API spec), Day 8 (diagrams)

Member	Learn (specific, limited)	Build	Output by Day 10	Deadline
Nidhi	Basic HTML form elements (input types, labels). How to read a test result table. Markdown tables.	2 test HTML pages (login form with PII + search page). Test scenario matrix. Demo narration script. Collect screenshots during testing.	2 test pages + scenario matrix + 60s demo script	Day 4 (pages), Day 9 (demo script)

SECTION 3 — 10-DAY ROADMAP

DAY 1 — Setup + Align

Goal: Everyone has environment ready. Architecture is frozen for 10 days. No scope creep.

Member	Task
Ayush	Create GitHub repo with folder structure (see Section 5). Install Playwright (<code>pip install playwright, playwright install chromium</code>). Write <code>hello_playwright.py</code> that opens <code>google.com</code> and takes a screenshot.
Shaurya	On Legion: clone <code>llama.cpp</code> , compile with CUDA (<code>cmake -DGGML_CUDA=ON</code>). Download <code>Qwen3.5-4B-Instruct-Q4_K_M.gguf</code> + <code>mmproj</code> file from HuggingFace. Run <code>llama-server</code> with both files. Confirm it starts.
Aditi	Read Qwen-VL model card. Understand: vision encoder → projector → LLM. Write 3 candidate grounding prompts on paper (before testing).
Abhishek	Install <code>mediapipe</code> , <code>opencv-python</code> , <code>Pillow</code> . Write 10 lines that detect a face in a test image using MediaPipe. Confirm it works.
Himanshu	Design the API endpoints on paper. Read the <code>00_EXECUTIVE_VERDICT.md</code> and <code>02_REFERENCE_ARCHITECTURE.md</code> files already created. Write a 1-paragraph explanation of our system for a non-engineer.
Nidhi	Create <code>test_pages/login_form.html</code> — a realistic-looking login page with username, password, email fields and a placeholder face <code></code> . Push to repo.
Checkpoint	Team call (30 min): Shaurya screenshares <code>llama-server</code> running. Ayush screenshares Playwright screenshot. Nidhi shares the test page in browser.
End of Day 1	Environment working for all. Repo exists.  Architecture frozen.

DAY 2 — First VLM Inference + Browser Capture

Goal: VLM answers a question about a screenshot. Browser capture pipeline works.

Member	Task
Ayush	Write <code>browser/screenshot.py</code> : opens <code>test_login.html</code> , captures full-page screenshot (1280×720 PNG), saves to <code>/tmp/</code> . Write <code>browser/dom_extractor.py</code> : calls <code>page.accessibility.snapshot()</code> , saves JSON.
Shaurya	Write <code>models/vlm_client.py</code> : Python function <code>ask_vlm(image_path, prompt) → str</code> . Calls llama-server via httpx using OpenAI-compatible format. Test with a screenshot of google.com and prompt "describe this page in one sentence".
Aditi	Take Nidhi's <code>test_login.html</code> screenshot manually. Send to Shaurya's <code>vlm_client</code> with 3 different prompts: "What UI elements are visible?", "Where is the Login button? Give coordinates.", and one of her own. Record what the VLM returns for each.
Abhishek	Write <code>privacy/dom_scanner.py</code> : takes DOM JSON → returns list of sensitive fields: <code>[{"type": "password", "tag": "input", "attr": "type=password"}, ...]</code> . Test on Nidhi's page DOM.
Himanshu	Define the JSON interface for <code>grounding response</code> (what Aditi's module returns to Ayush). Share with team via Slack/WhatsApp. Write it as a comment in the repo.
Nidhi	Create <code>test_pages/search_page.html</code> — a simple page with a search bar, a submit button, and some fake user info (email in footer, phone in "About" section).
Checkpoint	Ayush + Shaurya call: confirm screenshot → <code>vlm_client</code> → text response works.
End of Day 2	VLM responds to a screenshot. Browser screenshot captured.  DOM extraction working.

DAY 3 — First Grounding Result

Goal: VLM returns a bounding box for a named element. Even if imperfect, it must return coordinates.

Member	Task
Ayush	Write <code>agent/executor.py</code> : function <code>execute_action(action_type, x, y) → calls <code>page.mouse.click(x, y)</code></code> . Test manually with hardcoded coordinates on <code>test_login.html</code> .
Shaurya	Optimize llama-server: set <code>--ctx-size 2048</code> , experiment with <code>--n-gpu-layers 99</code> . Monitor VRAM with <code>nvidia-smi</code> . Document: VRAM at rest vs during inference.

Member	Task
Aditi	Build <code>perception/grounding.py</code> : <code>function ground_element(image_path, description) → {bbox, confidence}</code> . Uses <code>vlm_client</code> internally. Parses the VLM's text response to extract <code>[x, y, w, h]</code> . Test on "Login button", "Search button", "Email field" — record results.
Abhishek	Write <code>privacy/regex_engine.py</code> : patterns for email, Indian phone (+91...), Aadhaar (12-digit), password field text (looking for "*" chars). Test against Nidhi's page text content.
Himanshu	Write <code>docs/api_spec.md</code> : 3 endpoint specs with request/response JSON examples. Share with Ayush for FastAPI implementation.
Nidhi	Write test scenario matrix for <code>test_login.html</code> : list every element, expected detection (PII or not), expected grounding result. Share with Aditi + Abhishek.
Checkpoint	Aditi shares her grounding results for 3 elements. Team judges: is the bbox roughly correct? (within $\pm 50\text{px}$ is a pass for day 3).
End of Day 3	Grounding returns coordinates for at least 2/3 elements. ☒ Executor can click a hardcoded coordinate.

DAY 4 — Connect Grounding → Click

Goal: Agent takes a task, grounds the element, clicks it. First true end-to-end slice.

Member	Task
Ayush	Write <code>agent/loop.py</code> (minimal version): takes <code>task_string</code> → takes screenshot → calls <code>ground_element()</code> → calls <code>execute_action()</code> → takes new screenshot → returns before/after images. Wire up: <code>python agent/loop.py "Click the Login button"</code> .
Shaurya	Write verification: <code>models/verification.py</code> : compare before/after screenshots (simple pixel diff or check if page content changed). Returns <code>{changed: True/False}</code> .
Aditi	Improve grounding prompt based on Day 3 results. Add coordinate parsing robustness: handle VLM saying "around (340, 280)" or "x=340, y=280" or JSON format. Deliver <code>ground_element()</code> function that reliably parses all these.
Abhishek	Write <code>privacy/face_detector.py</code> : MediaPipe BlazeFace on screenshot → returns list of face bounding boxes. Test on Nidhi's page (must have a visible face in the HTML).
Himanshu	Write <code>server/api.py</code> (stub): FastAPI app with endpoint <code>POST /task</code> that accepts <code>{task: string}</code> . Returns <code>{status: "ok"}</code> for now. Ayush will fill in the real logic.

Member	Task
Nidhi	Manually test Nidhi's test pages in Chrome. Document: "When I click Login, what should happen?" Write expected outcome for each demo task. Prepare 2 dummy face photos (royalty-free) for the test pages.
Checkpoint	FIRST LIVE DEMO: Ayush runs <code>loop.py</code> "Click the Login button". Team watches. Does it click anywhere near the button? Record the video (.mp4).
End of Day 4	Agent clicks a browser element based on VLM grounding output. Even if imperfect, the connection exists.

DAY 5 — First Complete End-to-End Run

Goal: Full pipeline runs. Screenshot → VLM → bbox → click → verify. This is MVP v0.

Member	Task
Ayush	Wire <code>loop.py</code> into <code>api.py</code> : <code>POST /task</code> now runs the agent loop. Add WebSocket <code>GET /status</code> that streams agent step updates. Write <code>frontend/index.html</code> : a minimal single-page control panel — text input for task, button to submit, area to show step logs, area to show before/after screenshots.
Shaurya	Ensure <code>vlm_client</code> handles errors gracefully (timeout, empty response). Add retry logic (max 2 retries). Test inference latency across 10 screenshots — record avg/min/max.
Aditi	Create <code>perception/test_grounding.py</code> : runs grounding on 10 saved test screenshots, reports success rate. Fix any remaining coordinate parsing bugs. Hand off <code>ground_element()</code> — it's done.
Abhishek	Write <code>privacy/redactor.py</code> : takes screenshot + list of regions → draws black rectangles over DOM-detected PII fields + blurs face regions. Outputs sanitized PNG. Test: input <code>test_login.html</code> screenshot → output should have password/email masked and face blurred.
Himanshu	Create Mermaid architecture diagram: <code>Browser</code> → <code>Screenshot</code> → <code>VLM</code> → <code>BBox</code> → <code>Click</code> → <code>Verify</code> . Create data flow diagram: what data moves at each step. Push to <code>docs/</code> .
Nidhi	Run full demo manually 3 times on <code>test_login.html</code> . Write what worked and what broke. Collect: 1 screenshot of the browser before action, 1 after.
Checkpoint	End of Day 5 team demo: Full pipeline running. All 6 watch together. Record video.
End of Day 5	Running MVP: <code>task</code> → <code>VLM ground</code> → <code>click</code> → before/after screenshot shown in UI. This is the minimum the team can show at internal judging if everything else fails.

DAY 6 — Add PII Detection + Redaction

Goal: Original vs sanitized screenshot side-by-side visible in the demo UI.

Member	Task
Ayush	Update <code>loop.py</code> : after taking screenshot, call Abhishek's <code>redact()</code> . Store both original and sanitized. Update frontend: show 2 image panels: "Original" (with PII visible + highlighted regions) and "Sanitized" (masked). Add a simple text note: "PII detected: password field, email, face photo — redacted locally."
Shaurya	Add <code>GET /metrics</code> endpoint: returns <code>{vlm_ms, redaction_ms, vram_mb}</code> . Log these after every agent run. Add VRAM monitoring call (<code>nvidia-smi --query-gpu=memory.used --format=csv,noheader</code>).
Aditi	Help debug any grounding failures found during Day 5 testing. Try "Set-of-Mark" prompting if standard grounding is still shaky: number all elements in the screenshot, ask VLM "which number is the Login button?" (This is a stretch — only if standard grounding needs help.)
Abhishek	Finalize <code>redactor.py</code> . Handle edge case: if face detection finds no face, don't crash — still mask DOM-detected fields. Write a simple test: run redaction on all images in <code>tests/fixtures/</code> , save outputs, check manually.
Himanshu	Write <code>docs/privacy_flow.md</code> : 1 page explaining what is redacted, why, and what never leaves the device. This becomes a PPT slide. Write the "Privacy Guarantee" statement: what we claim and what the limits are.
Nidhi	Collect PPT evidence screenshots: raw page with PII visible, sanitized page with PII masked. Start building PPT slide skeleton with Himanshu.
Checkpoint	Ayush shares screen: runs demo, side-by-side panel shows original vs redacted. Everyone verifies password is blacked out and face is blurred.
End of Day 6	Original vs sanitized screenshot visible in real demo UI. PII is masked for DOM-detected fields and face.

DAY 7 — Remote Model Integration (Real or Mocked)

Goal: Show data flow to remote model. Real API if possible, clearly labeled mock if not.

Member	Task
Ayush	Add remote model step to <code>loop.py</code> : after redaction, call Shaurya's <code>remote_client</code> . If it returns a response, render it in the UI. Add clear label: "Remote Model Response (sanitized data only)" or "⚠️ MOCKED — Remote Model".

Member	Task
Shaurya	Write <code>models/remote_client.py</code> : send sanitized screenshot + task to GPT-4o or Qwen3.8-Max API. Parse response (it will say something like "Click the Login button at approximately center-right of the form"). If real API is unavailable, return a hardcoded mock response — clearly mark it in code as <code># MOCKED</code>.
Aditi	Run grounding experiment on <code>search_page.html</code> (second test page). Does the same prompt work? Document differences. Flag which page works better and why.
Abhishek	Add regex detection to <code>privacy/regex_engine.py</code> : scan DOM text for Aadhaar pattern, email, phone. Show detected text in the "PII found" panel. Run on both test pages.
Himanshu	Finalize <code>docs/api_spec.md</code> . Write data flow: diagram showing what bytes flow from local to remote and back. Specifically: remote model gets <code>{task, sanitized_screenshot}</code> , never raw screenshot. This is a PPT talking point.
Nidhi	Draft the 60-second demo script (Section 8 of this document as template). Practice saying it once out loud. Time it.
Checkpoint	Full pipeline demo: task → ground → click → redact → (real or mocked) remote call → response shown in UI. Record video.
End of Day 7	Full pipeline works with remote step present (real or mocked — clearly labeled).

DAY 8 — Integration Polish + Stability

Goal: System runs 5 times in a row without crashing. Frontend looks clean enough to screenshot.

Member	Task
Ayush	Fix any integration bugs from Day 7. Polish frontend: make the 3-panel layout (control panel)
Shaurya	Run the full pipeline 10 times back-to-back. Fix crashes. Record latency from each run. Calculate average. Make sure llama-server doesn't need restarting between runs.
Aditi	Write <code>perception/benchmark.py</code> : run grounding on all test images, output accuracy table (element, predicted bbox, was it correct? Y/N). Produce this table as <code>results/grounding_accuracy.md</code> .
Abhishek	Run redaction on both test pages, save results. Write <code>results/redaction_results.md</code> : for each page, list detected PII items + whether they were masked. Share with Nidhi for PPT.

Member	Task
Himanshu	Finalize architecture diagram and data flow diagram. Export as PNG for PPT. Write 2 PPT slides: system architecture + privacy claim. Review with team.
Nidhi	Collect all evidence screenshots (see Section 9). Organize them into PPT format. Run demo script 3 times with Ayush as operator.
Checkpoint	Full demo runs cleanly 3 times. All PPT evidence collected.
End of Day 8	System stable. Frontend presentable. All screenshots collected.

DAY 9 – Testing + Known Issues + Jury Prep

Goal: Know exactly what works, what doesn't, and have honest answers for both.

Member	Task
Ayush	Run Nidhi's test scenario matrix against the live demo. For each scenario: does it pass? Document failures honestly. Fix the top 2 most embarrassing failure cases.
Shaurya	Run VRAM profiling: document peak VRAM during operation. Run latency report: average per-step breakdown. Both go into PPT evidence.
Aditi	Finalize grounding accuracy table. Pick the best 3 results as visual evidence (annotated screenshots). Write honest assessment: "grounding works for X types of buttons, struggles with Y".
Abhishek	Write honest PII detection results: "We detect password fields, email fields, faces. We do NOT yet detect Aadhaar in image form. We DO detect Aadhaar in visible text via regex." This honesty is better than overclaiming.
Himanshu	Prepare jury Q&A answers (Section 10). Run mock jury: each team member answers 3 questions from the list.
Nidhi	Finalize PPT. Finalize demo script. Create backup: a screen-recording of the working demo saved as .mp4 (in case live demo fails).
Checkpoint	Mock internal jury run (30 min). All 6 members sit together. 2 people (Himanshu + Aditi) role-play as judges and ask questions.
End of Day 9	Known issues documented. Honest accuracy numbers in hand.  PPT complete.  Backup video ready.

DAY 10 — Demo Stabilization + Final Rehearsal

Goal: Demo runs reliably. Everyone knows their role. Ready to walk in.

Member	Task
Ayush	Demo rehearsal ×3. Fix any last-minute display bugs. Ensure test pages load instantly (no network dependency).
Shaurya	Pre-warm llama-server: run first inference 10 min before demo. This clears cold-start latency. Verify GPU is in performance mode.
Aditi	Review jury Q&A answers for VLM-related questions. Be ready to draw the VLM pipeline on a whiteboard.
Abhishek	Review jury answers for privacy/redaction questions. Be ready to answer: "what if redaction misses something?"
Himanshu	rehearse PPT presentation. Ensure diagrams are legible from distance. Know the data flow cold.
Nidhi	Rehearse demo narration. Have printed backup of demo script. Know the 60-second flow by heart.
Checkpoint	Full dress rehearsal: PPT presented + live demo run + jury Q&A (15 min).
End of Day 10	Ready to present.

SECTION 4 — INDIVIDUAL LEARNING PLAN

AYUSH

Learn:

- playwright Python: `page.goto()`, `page.screenshot()`, `page.mouse.click(x, y)`, `page.accessibility.snapshot()`
- Python `asyncio` basics: `async def`, `await`, running `async` from `sync` context
- FastAPI: `@app.post()`, `@app.websocket()`, `JSONResponse`
- `httpx.AsyncClient` for calling llama-server

Build: Agent loop + demo UI

Know by Day 10:

- "Our agent loop takes a screenshot, sends it to the local VLM, parses the bounding box, and executes a Playwright click at those coordinates."
- Walk through the code in `agent/loop.py` on screen.

SHAURYA

Learn:

1. llama-server CLI flags: `--model`, `--mmap`, `--port`, `--n-gpu-layers`, `--ctx-size`
2. OpenAI API format for vision: `{"role": "user", "content": [{"type": "image_url", ...}, {"type": "text", ...}]}`
3. `nvidia-smi` output: how to read VRAM usage
4. `httpx` for async HTTP requests
5. GGUF Q4_K_M: what "Q4" means (4-bit quantization, ~4x size reduction, small quality tradeoff)

Build: VLM inference server + Python client

Know by Day 10:

- "Our VLM runs on 3.8 GB of VRAM, which is well within the 8 GB budget. Q4_K_M quantization reduces the model from 8 GB to 3.5 GB with minimal accuracy loss."
- Show `nvidia-smi` output during inference.

ADITI

Learn:

1. What a vision encoder does (ViT turns image → patch tokens → embedding)
2. What grounding means: model maps text description → pixel coordinates
3. Bounding box formats: `[x, y, w, h]` vs `[x1, y1, x2, y2]`, normalized (0.0–1.0) vs pixel
4. Prompt engineering for grounding: specificity matters ("click the blue Login button in the form" > "find the button")
5. How to calculate IoU (intersection-over-union) to measure grounding accuracy

Build: `ground_element()` function + accuracy report

Know by Day 10:

- "Grounding means the VLM takes the screenshot and our question, and returns pixel coordinates. We tested on 10 screenshots and achieved correct localization in X/10 cases."
- Show the annotated screenshots with predicted vs actual bounding boxes.

ABHISHEK

Learn:

1. Python `re` module: `re.findall()`, `re.compile()`, character classes, `\d`, `\b`
2. DOM attribute scanning via Playwright: `page.query_selector_all('input')`, reading `.type`, `.name`, `.id`
3. OpenCV: `cv2.rectangle()` to draw a black box, `cv2.GaussianBlur()` for face blur

4. MediaPipe face detection: `mp.solutions.face_detection.FaceDetection`, reading `detection.location_data.relative_bounding_box`

Build: PII detector + redactor (DOM scan + regex + face blur)

Know by Day 10:

- "We detect PII in 3 ways: DOM attribute scanning catches password/email fields instantly. Regex catches patterns like Aadhaar numbers and Indian phone numbers in page text. MediaPipe detects face photos and we Gaussian-blur them."
 - Show before/after redacted screenshot.
-

HIMANSHU

Learn:

1. HTTP request/response: method, headers, body, status code
2. JSON format (read and write basic JSON)
3. Mermaid diagram syntax: `graph LR, A --> B, subgraph`
4. How to explain: "what is an API?" and "what is a REST endpoint?" in plain English
5. The difference between local and remote processing in our architecture

Build: API spec + architecture + data flow diagrams

Know by Day 10:

- Draw the full data flow on a whiteboard: "Browser → Screenshot → VLM → BBox → Click → Redact → Sanitized → Remote."
 - Explain why raw PII never leaves the device.
 - Defend the API design: "We have a `/task` endpoint that takes a task string. It returns step-by-step progress via WebSocket."
-

NIDHI

Learn:

1. HTML: `<form>`, `<input type="password">`, `<input type="email">`, `<img>`, `<button>`
2. How to read a test result table (pass/fail columns)
3. Markdown table syntax
4. What PII means and 4 examples relevant to our project (password, email, Aadhaar, face)

Build: 2 test pages + scenario matrix + demo script

Know by Day 10:

- Narrate the 60-second demo without reading from paper.
 - Explain (1 sentence each): What is PII? What does the agent do? Why is privacy important here?
-

SECTION 5 – INTEGRATION CONTRACT

These are the function signatures that each member must respect. **Do not change them without team agreement.**

Aditi → Ayush (Grounding)

```
# perception/grounding.py
def ground_element(image_path: str, description: str) -> dict:
    """
    Returns:
    {
        "element": "Login button",
        "bbox": [x, y, width, height],    # pixel coordinates
        "confidence": 0.91,
        "raw_response": "..."           # VLM raw text (for debugging)
    }
    Returns {"bbox": None, "confidence": 0.0} if not found.
    """
```

Abhishek → Ayush (Redaction)

```
# privacy/redactor.py
def redact(image_path: str, dom_json: dict) -> dict:
    """
    Returns:
    {
        "sanitized_image_path": "/tmp/sanitized_20260908.png",
        "detected_pii": [
            {"type": "password_field", "bbox": [x, y, w, h], "source": "dom"},
            {"type": "email_field",     "bbox": [x, y, w, h], "source": "dom"},
            {"type": "face",            "bbox": [x, y, w, h], "source": "mediapipe"},
            {"type": "aadhaar_text",    "bbox": None,         "source": "regex", "value": "REDACTED"}
        ]
    }
    """
```

Shaurya → Ayush (VLM Inference)

```
# models/vlm_client.py
def ask_vlm(image_path: str, prompt: str) -> str:
    """
    Sends image + prompt to llama-server.
    Returns raw text response from the model.
    Raises TimeoutError if server doesn't respond in 10s.
    """
```

Shaurya → Ayush (Remote Model)

```
# models/remote_client.py
def ask_remote(sanitized_image_path: str, task: str) -> str:
    """
    Sends SANITIZED image + task to remote model API.
    Returns reasoning text.
    If MOCKED, returns: "Based on the interface, click the Login button."
    Add comment: # MOCKED - replace with real API call
    """
```

Ayush → Frontend (WebSocket)

```
// Stream these events via WebSocket as the agent runs:
{"step": "screenshot", "status": "done", "ms": 45}
{"step": "grounding", "status": "done", "ms": 380, "bbox": [342, 280, 120, 40]}
{"step": "action", "status": "done", "ms": 95, "action": "click(402, 300)"}
{"step": "redaction", "status": "done", "ms": 48, "pii_count": 3}
{"step": "remote", "status": "mocked", "ms": 0, "response": "Click Login button"}
{"step": "complete", "status": "done", "total_ms": 568}
```

Himanshu — API Endpoints (Ayush implements)

```
POST /task
  Body: {"task": "Click the Login button", "url": "http://localhost:8080/test_login.html"}
  Response: {"run_id": "abc123", "status": "queued"}

GET /results/{run_id}
  Response: {"status": "complete", "original": "...", "sanitized": "...", "metrics": {...}}

WS /status
  Streams JSON events (see above format)
```

SECTION 6 — DAILY INTEGRATION RULE

The Rule

No more than 1 day of solo work without pushing to the repo and sharing output with the team.

Specifically:

- Ayush and Shaurya: push before sleeping, every single night.
- Aditi, Abhishek: push your module the moment it passes your own test.
- Himanshu, Nidhi: push docs/pages the day they're written.

Git Strategy (Keep It Simple)

```
main ← only stable, tested code
dev ← integration branch (everyone merges here)
├── feat/ayush-agent-loop
├── feat/shaurya-vlm-server
├── feat/aditi-grounding
├── feat/abhishek-redaction
├── feat/himanshu-api-docs
└── feat/nidhi-test-pages
```

- Each member works on their branch.
- When something works, they open a PR to `dev`.
- Ayush or Shaurya reviews + merges (takes < 5 min — this is a prototype, not production).
- `dev` → `main` only when a live demo is confirmed working.

Integration Meetings

- **Days 1–5:** 30-min call every evening. Ayush screenshares the running system.
- **Days 6–10:** 30-min call every evening + one full dress-rehearsal on Day 9.

No Long Diverging Work

If Aditi is on Day 3 and her grounding code isn't usable by Ayush yet → Ayush uses a **hardcoded mock** `ground_element()` that returns `{bbox: [342, 280, 120, 40]}` and keeps building. Aditi catches up. The pipeline never stalls.

SECTION 7 — WHAT TO CUT

Do not touch any of the following in these 10 days. Even if it sounds useful.

Cut This	Why
Agent conversation memory / history	Adds complexity, irrelevant for single-task demo
Any database (SQLite, Postgres, anything)	JSON files and in-memory state are sufficient
User authentication for the demo app	No user management needed — this is a local prototype
Docker or containerization	Direct Python environment is faster to debug
Sophisticated routing logic (local vs remote)	In prototype: always go remote after redaction. Routing is a Day 30+ concern.
Multi-step autonomous planning	Single-step tasks are enough to demonstrate the concept

Cut This	Why
Model fine-tuning	Not feasible in 10 days; pre-trained Qwen3.5-4B is sufficient
Custom frontend framework (React, Vue)	Vanilla HTML + CSS is enough for internal judging
Fancy animations or loading spinners	Waste of Ayush's time
CI/CD pipelines	Run tests manually
GLiNER NER	Out of scope for 10 days
PaddleOCR	VLM's built-in OCR is sufficient for prototype
Wireshark proof of no PII	Powerful but too complex for Day 10; verbal explanation + architecture diagram suffices
More than 2 test pages	Nidhi's 2 pages are enough
Production error handling	Basic try/except is fine
API rate limiting, CORS headers	Not needed for local demo

SECTION 8 – FINAL DEMO SCRIPT (60–90 seconds)

Who operates: Ayush (at keyboard), Nidhi (narrates), Shaurya (answers technical questions)

Setup before demo: llama-server pre-warmed. Test page open. Demo UI open in browser. Shaurya has nvidia-smi open in terminal.

[0:00]

Nidhi: "Browser agents automate tasks on websites. But they have a privacy problem — they send your screen, which contains passwords and personal data, to cloud servers. We solve this."

- [Ayush shows the test login page in the browser — username, password, email, a face photo visible]*

[0:10]

Nidhi: "We give the agent a task."

- [Ayush types into the control panel: "Click the Login button" → hits Run]*

[0:15]

Nidhi: "The agent captures the page and sends it to a lightweight AI model running locally — on this laptop. No data leaves the machine yet."

- [WebSocket panel shows: Screenshot: 45ms → Grounding: 380ms → Bounding box found: [342, 280, 120, 40]]*

[0:25]

Nidhi: "The model locates the Login button — here — and the agent clicks it."

- [Browser visibly clicks the button — page changes to "Welcome!" or similar]*

[0:35]

Nidhi: "Now — privacy. Before anything leaves the device, our system scans the original screenshot."

- [Left panel: Original screenshot — password field, email highlighted in red boxes, face circled in green]*

Nidhi: "Password field detected via DOM. Email detected via DOM. Face detected via computer vision."

- [Right panel appears: Sanitized screenshot — password blacked out, email blacked out, face blurred]*

Nidhi: "Redacted. Locally. In 45 milliseconds."

[0:50]

Nidhi: "Only the sanitized version is sent to a remote server for complex reasoning if needed. The raw data never leaves this machine."

- [Text panel: "Remote model received: sanitized screenshot. Raw PII transmitted: 0 items."]*

[1:00]

Nidhi: "Local inference: 380ms. Redaction: 45ms. VRAM used: 3.8 GB of 8 GB."

- [Shaurya points to nvidia-smi showing 3.8 GB]*

Nidhi: "This is privacy-preserving browser automation — on-device, measurable, and real."

SECTION 9 — PPT EVIDENCE (collect these 8 items)

#	Evidence	Who Collects	When
1	Architecture diagram — Mermaid export showing full pipeline as PNG	Himanshu	Day 5
2	Local VLM responding — screenshot of terminal showing llama-server running + inference latency output	Shaurya	Day 3
3	Grounding visualization — screenshot of the original page with the predicted bounding box drawn on the target element (red box around Login button)	Aditi	Day 4
4	Original vs Sanitized — side-by-side screenshot: left has visible password/email/face, right has all three masked	Ayush/Abhishek	Day 6
5	Working browser action — before/after screenshots: page before click + page after click (showing form submitted)	Ayush	Day 5
6	VRAM chart — nvidia-smi showing GPU memory during demo (3.x GB / 8 GB)	Shaurya	Day 8
7	Latency table — simple table: inference avg Xms, redaction avg Yms, total pipeline avg Zms	Shaurya + Abhishek	Day 8
8	PII detection results — table: PII Type	Detected?	Method (source: DOM/regex/vision)

Do not create more than these 8 pieces of evidence. Pick the best screenshot for each. Quality > quantity.

SECTION 10 — JURY PREPARATION (15 Questions)

#	Question	Answers	Who Answers
1	Why on-device?	Privacy: screenshots contain PII. On-device = PII never transmitted. Also lower latency and no cloud dependency.	Himanshu
2	Why a VLM instead of traditional automation?	Traditional automation uses brittle CSS selectors that break when a site updates. VLMs understand ANY webpage visually, like a human would — no pre-written selectors needed.	Aditi

#	Question	Answers	Who Answers
3	What is grounding?	Grounding maps a text description ("the Login button") to exact pixel coordinates on screen. Without it, the agent can understand the page but can't act on it.	Aditi
4	Why not send the raw screenshot to the cloud?	Screenshots contain passwords, Aadhaar numbers, faces. Sending them to a cloud server is a serious privacy violation. Our system redacts all PII before anything leaves the device.	Himanshu
5	How does redaction work?	3 layers: DOM attribute scan (catches password/email input fields by HTML type), regex scan (catches Aadhaar, phone, email patterns in page text), face detection (MediaPipe BlazeFace detects faces and we blur them).	Abhishek
6	What if redaction misses something?	Good question — no system is perfect. We have a post-redaction re-scan that checks the sanitized image for any remaining PII. If found, we block the outbound request. Our benchmark shows X% recall on our test set.	Abhishek
7	How lightweight is "lightweight"?	Qwen3.5-4B with Q4_K_M quantization — 4 billion parameters, uses 3.5 GB of VRAM. This runs on an RTX 3060/4060 — a normal gaming laptop. No server or datacenter needed.	Shaurya
8	How much VRAM does it use?	3.8 GB peak during inference, out of 8 GB available. We have headroom for face detection and OCR simultaneously. [Show nvidia-smi screenshot.]	Shaurya
9	What is the latency?	Local VLM inference: ~380ms average. Redaction: ~45ms. Total pipeline per step: ~600ms. Web browsers add ~100ms. Compare to cloud-only: 2–5s per step, with all PII exposed.	Shaurya
10	What is actually implemented vs mocked?	Honest answer: Implemented: browser screenshot, VLM inference (real local model), grounding, Playwright click, DOM PII scan, regex PII scan, face detection, image redaction. Mocked: remote model response (we used a hardcoded reply for the demo, but the API call structure is real).	Ayush
11	What is novel about your approach?	Most browser agents send raw screenshots to the cloud. We are one of few that make privacy a measurable property: we benchmark PII detection recall, we prove 0 raw PII transmitted, we demonstrate it with real numbers.	Himanshu
12	How does this scale into the final SIH	The core modules are already designed to plug together. The next steps are: better grounding accuracy, multi-step	Ayush

#	Question	Answers	Who Answers
	solution?	planning, GLiNER NER for names/addresses, and a polished UI. The architecture we've built is the foundation.	
13	Why Qwen3.5-4B specifically?	After evaluating SmolVLM (2.2B — too weak for grounding), MiniCPM-V (8B — too tight on VRAM at 7–8 GB), and Qwen3.5-4B (4B, 3.5 GB VRAM, strong grounding, Apache 2.0 license), Qwen3.5-4B gave the best accuracy within our 8 GB constraint.	Aditi
14	What happens if the local model gives wrong coordinates?	The agent executes the click, takes a new screenshot, and checks if the page changed as expected. If not, it retries with a more specific prompt — up to 2 retries before reporting failure.	Ayush
15	Why not just parse the DOM for everything instead of using a VLM?	DOM works for standard HTML elements, but modern websites use custom components, canvas, shadow DOM, and JavaScript-rendered content that has no reliable DOM structure. The VLM sees what the user sees — the rendered page — and is robust to these variations. DOM is our supplement, not our foundation.	Aditi

FINAL SECTION — GO/NO-GO CHECKLIST

By Day 3

- ☐ Shaurya's llama-server running and responding to an image+prompt via curl
- ☐ Ayush's Playwright script takes a screenshot of test_login.html
- ☐ Abhishek's regex detects an email pattern in a text string
- ☐ Nidhi's 2 test HTML pages exist in the repo

By Day 5

- ☐ agent/loop.py runs a full observe → ground → click → verify cycle (even if grounding is rough)
- ☐ Before/after screenshots captured automatically
- ☐ Aditi's ground_element() returns a bbox for at least 1 element correctly
- ☐ FastAPI server accepts POST /task and runs the agent

By Day 7

- ☐ Original vs sanitized screenshot visible side by side in the demo UI
- ☐ Face detection running on test page (blurring the face photo)
- ☐ Remote model step present in pipeline (real or clearly labeled mock)
- ☐ WebSocket streams agent step updates to frontend

By Day 9

- ☐ Demo runs 3 times without crashing
- ☐ All 8 evidence screenshots collected
- ☐ PPT draft complete
- ☐ Backup demo video recorded (.mp4)
- ☐ Every team member has practiced their jury questions

By Day 10

- ☐ Demo runs 5 times cleanly
- ☐ Narration rehearsed by Nidhi (timed: 60–90 seconds)
- ☐ Every member can answer their assigned jury questions without reading
- ☐ Backup plan confirmed: if live demo fails, switch to video immediately

THE ABSOLUTE MINIMUM DEMO

If everything else fails, this must still work:

You open `test_login.html` in a browser. You run `python agent/loop.py "Click the Login button"`. The browser clicks the Login button. You show the before screenshot and the after screenshot side by side. You say: *"Local VLM grounded the Login button at these coordinates. The agent clicked it. No data left the device."*

That's it. That is sufficient to demonstrate the core concept and earn the internal selection.

Everything else — redaction, hybrid routing, demo UI, benchmarks — makes it more convincing. But the absolute core is: **local VLM sees page → grounds element → browser acts.**

Don't lose sight of that.